Project Development Phase
Coding & Solution

Date	10 July 2026
Team ID	SWTID-2026-6119
Project Name	Credit Card Approval Prediction System
Maximum Marks	4 Marks

Overview:

This document summarizes the actual working solution built for the Credit Card Approval Prediction System: a Jupyter notebook that trains and compares four ML models on a real credit approval dataset, and a Flask web application that serves the best model for live predictions.

1. Dataset Used:

The **UCI Credit Approval dataset** (690 real, anonymized credit card application records) was used for training and evaluation. It contains a mix of demographic and financial features (Age, Income, Debt, YearsEmployed, CreditScore, etc.) and a binary target (Approved / Rejected), closely matching the real-world structure of a credit approval decision.

2. Data Preprocessing (implemented in EDA_and_Model_Training.ipynb):

- Missing values ('?' in the raw data) converted to NaN, then imputed - mean for numeric columns, mode for categorical columns.
- Duplicate records removed.
- Categorical features label-encoded; low-value features (DriversLicense, ZipCode) dropped.
- Numeric features scaled using StandardScaler.
- Data split into training (80%) and testing (20%) sets with stratification to preserve class balance.

3. Model Training & Comparison:

Model	Accuracy	Precision	Recall	F1-Score
Logistic Regression	0.891	0.859	0.902	0.880
Decision Tree	0.877	0.833	0.902	0.866
Random Forest	0.906	0.929	0.852	0.889
XGBoost (Selected)	0.906	0.900	0.885	0.893

XGBoost was selected as the deployed model - it achieved the highest F1-Score (0.893), balancing Precision and Recall better than the alternatives. F1-Score was prioritized over raw Accuracy because both false approvals (risk to the bank) and false rejections (lost creditworthy customers) carry real business cost.

4. Model Persistence:

The trained XGBoost model, the fitted label encoders, the fitted StandardScaler, and the ordered list of feature columns are all saved using **joblib** (model.pkl, encoders.pkl, scaler.pkl, feature_columns.pkl), so the Flask application can load them once at startup without retraining.

5. Flask Application (Flask/app.py):

- **GET /** - renders the application form (index.html) with dropdowns for categorical fields and number inputs for financial fields.

- **POST /predict** - validates the submitted form, converts it into a model-ready feature vector using the saved encoders/scaler, runs the model, and renders the result page with the Approved/Rejected outcome and the model's confidence score.
- **404 handler** - renders a custom, on-brand error page instead of Flask's default error screen for any invalid route.
- All model artifacts are loaded once at application startup (not per-request) for performance.

6. Sample Code Snippet (Prediction Logic):

```
@app.route("/predict", methods=["POST"])
def predict():
    form_data = request.form.to_dict()
    X = build_feature_vector(form_data)
    prediction = model.predict(X)[0]
    probability = model.predict_proba(X)[0][1]
    result = "Approved" if prediction == 1 else "Rejected"
    return render_template("result.html", result=result,
                           confidence=round(probability * 100, 2))
```